



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES
DEPARTAMENTO DE INGENIERIA DE SOFTWARE
DESARROLLO DE SOFTWARE PARA PLATAFORMA MÓVIL



PLAN DEL PROYECTO

Aplicación móvil Android para gestión integral de un taller mecánico

Facilitador:

Jorge Cisneros

Integrantes:

Camaño, Edward	8-1010-515
González, Emanuel	8-1022-2376
Hooker, Gonzalo	8-1015-933
Hou, Edwin	8-1021-1916
Lezcano, Fernando	8-1013-1857
Madrid, María	8-1021-474

Grupo: 1SF241

I SEMESTRE, 2026

Índice

Resumen del proyecto	5
Acta de constitución del proyecto	6
Objetivo del proyecto	6
Justificación	6
Restricciones y supuestos	7
Restricciones	7
Supuestos	7
Criterios de éxito	7
Propuesta de valor y caso de negocio	8
Propuesta, funcionalidad y objetivo del prototipo	8
Autenticación y gestión de sesión	8
Inicio	9
Catálogo, carrito y compras	9
Solicitud y seguimiento de servicio	9
Perfil del cliente	10
Panel de administración (mismos módulos, controles ampliados)	10
Dashboard	10
Notificaciones en tiempo real	10
Sector comercial al que va dirigido	10
Público al que está orientado	11
Beneficios para la sociedad, el público y el sector comercial	11
Beneficio social	11
Beneficio para el público (cliente final)	11
Beneficio comercial (el negocio/taller)	12
Costo del proyecto	12
Declaración de alcance y especificación funcional	12
Incluido en el alcance	12
Fuera del alcance	13
Especificación funcional detallada por módulo	13
Autenticación y sesión	13
Vehículos	13
Carrito de compras	14
Pedidos y facturación	14

Servicios (solicitudes de reparación)	14
Productos e inventario	14
Contenido de Inicio	14
Registro de requisitos	15
Registro de interesados	16
Estructura de desglose del trabajo (EDT)	16
Equipo del proyecto y matriz RACI	17
Roles	17
Matriz RACI	18
Gestión del proyecto: Metodología Scrum	18
Por qué Scrum	19
Roles Scrum	19
Ceremonias aplicadas	19
Definition of Done	19
Product Backlog priorizado (muestra representativa)	20
Narrativa sprint a sprint: planeado vs. entregado	20
Sprint 1 (2 – 8 de julio)	20
Sprint 2 (9 – 12 de julio)	21
Plan de ejecución del proyecto	21
Entorno y herramientas	21
Flujo de ejecución de cada historia	21
Gestión de cambios durante la ejecución	22
Cronograma del proyecto	22
Presupuesto y plan de gestión de costos	23
Presupuesto detallado	23
Plan de gestión de costos	24
Registro de riesgos	24
Registro de problemas	25
Registro de decisiones	26
Plan de comunicación	27
Arquitectura de software y patrones de diseño	27
Vista general de la arquitectura	27
Patrón MVVM (Model-View-ViewModel)	28
Patrón Repository	29

Singleton	29
Tipado de rutas con clases selladas (type-safe navigation)	29
Inyección de dependencias manual (constructor injection)	30
Patrón Adaptador: doble camino de consumo del mismo API	30
Seguridad: JWT + Row Level Security	30
Flujo de datos unidireccional (StateFlow → Compose)	31
Modelo de datos	31
Sensores y hardware del dispositivo	32
GPS / ubicación	32
Cámara y Galería	33
Matriz de trazabilidad de requisitos	33
Checklist de calidad	34
Confirmación de cumplimiento del alcance	35
Informe final del proyecto	35
Checklist de cierre	36
Conclusión	36
Lecciones Aprendidas	37

Resumen del proyecto

Mecanic Hou es una aplicación móvil nativa para Android, desarrollada en Kotlin con Jetpack Compose, que digitaliza la operación completa de un taller mecánico desde la solicitud de servicios de reparación y la venta de repuestos, hasta la gestión administrativa del negocio como lo son el inventario, categorías (de inventario), órdenes de servicio y dashboard.

El proyecto responde a la creciente adopción de dispositivos móviles para gestionar servicios cotidianos, es decir, hoy en día un cliente espera poder solicitar un servicio, dar seguimiento a su vehículo y comprar repuestos desde su teléfono, sin depender de llamadas telefónicas o visitas presenciales para cada consulta. Del lado del negocio, el taller gana visibilidad en tiempo real de su operación mostrando que vehículos están en proceso, qué productos tienen stock bajo, y cómo se comportan sus ventas mes a mes.

La aplicación se construyó sobre una arquitectura cliente–servidor, en la misma el cliente Android consume el backend como servicio (Supabase: Postgres, autenticación, almacenamiento y tiempo real) a través de un API REST, con un módulo específico (gestión de categorías) consumido explícitamente mediante Retrofit para demostrar el dominio de los cuatro métodos HTTP (GET, POST, PUT, PATCH). El equipo trabajó bajo la metodología ágil Scrum, en dos sprints de una semana cada uno, tras una fase breve de planificación.

Este documento consolida, en un solo lugar, todos los documentos de proyecto iniciando por la propuesta de negocio y el registro de requisitos, hasta los registros de seguimiento (riesgos, problemas, decisiones), la arquitectura técnica, la trazabilidad entre requisitos y pruebas, y los checklists de cierre cubriendo tanto el componente contractual/comercial como el componente técnico.

Acta de constitución del proyecto

Nombre del proyecto	Aplicación móvil de gestión para un taller mecánico
Patrocinador / Cliente	Mecanic Hou
Gerente / Product Owner	Gonzalo Hooker
Fecha de inicio	29 de junio de 2026
Fecha de cierre del desarrollo	12 de julio de 2026
Fecha de entrega y sustentación	15 de julio de 2026
Duración efectiva de desarrollo	1 semana y media (11 días) en 2 sprints, tras 3 días de planificación
Plataforma	Android Nativo (Kotlin + Jetpack Compose), backend Supabase

Objetivo del proyecto

Diseñar, desarrollar y sustentar una aplicación Android funcional que resuelva un problema real de negocio (gestión de un taller mecánico), demostrando el dominio de autenticación segura basada en JWT, consumo de APIs REST, uso de sensores del dispositivo, y un panel de administración, dentro del marco de la metodología ágil Scrum.

Justificación

Los talleres mecánicos independientes en Panamá gestionan sus operaciones con herramientas que no fueron diseñadas para ese fin: agendas en papel, WhatsApp como sistema de seguimiento y cuadernos como único registro de inventario. Este modelo informal genera consecuencias directas sobre el negocio: pérdida de información por vehículo y cliente, tiempos de respuesta lentos, errores en la asignación de tareas y una nula visibilidad del dueño sobre el estado real de su taller en cualquier momento del día.

El problema no es falta de disposición tecnológica, sino ausencia de una herramienta pensada para su contexto.

El mecánico o el administrador de un taller no trabajan frente a un escritorio estos se mueven entre vehículos, bodegas y áreas de recepción. Una solución de escritorio o web convencional no encaja en ese ritmo. El smartphone es el dispositivo que ya

tienen consigo, y es además el canal desde el que ya gestionan buena parte de su negocio como los pagos con Yappy, coordinación por WhatsApp, consultas rápidas.

Una app móvil permite registrar órdenes de trabajo en el momento en que se abren, notificar al cliente cuando su vehículo está listo, asignar tareas a mecánicos en tiempo real y consultar el inventario desde cualquier punto del taller, sin depender de que alguien recuerde hacer algo manualmente.

Restricciones y supuestos

Restricciones

- **Tiempo:** el desarrollo efectivo comprende una semana y media antes de la sustentación, lo que delimita el alcance funcional a las características core de la aplicación.
- **Presupuesto:** la solución se construye sobre infraestructura de costo cero durante esta fase, utilizando la capa gratuita de Supabase. Esta decisión no es únicamente una limitante económica del equipo, sino que responde a una decisión estratégica acordada con el cliente: se establece un período de prueba de varios meses en producción real antes de escalar la infraestructura. Una vez validado el uso y la adopción del sistema, se realizará una migración hacia un plan de mayor capacidad que soporte un volumen creciente de usuarios, órdenes y datos concurrentes.
- **Equipo:** el proyecto cuenta con 6 integrantes con disponibilidad parcial, dado que todos cursan otras materias de forma simultánea durante el período de desarrollo.

Supuestos

- Se asume que el equipo dispone de al menos un dispositivo Android físico o emulador habilitado para la instalación y prueba de la aplicación.
- Se asume que existe conectividad a internet estable tanto durante el desarrollo como en la demostración en vivo, requisito necesario dado que la autenticación y el almacenamiento de datos dependen de los servicios en la nube de Supabase.
- Se asume que el cliente mantendrá el uso de la aplicación dentro de los límites del plan gratuito durante el período de prueba acordado, antes de proceder con la escalabilidad del servicio.

Criterios de éxito

Estabilidad técnica: la aplicación compila, se instala y se ejecuta sin errores en un dispositivo Android físico real. Ningún flujo principal termina en un cierre inesperado de la app ni deja la interfaz en un estado irreversible.

Flujo del cliente completo: un usuario con rol cliente puede, de principio a fin y sin intervención manual en la base de datos: crear su cuenta, iniciar sesión, explorar el catálogo, agregar productos al carrito, completar una compra y recibir su factura con ITBMS calculado, solicitar un servicio con los datos de su vehículo y evidencia fotográfica, consultar el estado de su orden en tiempo real, y cerrar sesión correctamente.

Flujo del administrador completo: un usuario con rol administrador puede gestionar el catálogo de productos y categorías, actualizar el stock, administrar las órdenes de servicio activas avanzando su estado, editar el contenido de la pantalla de inicio, y consultar el dashboard con reportes de ventas e inventario, todo desde la misma aplicación y sin acceder directamente a la base de datos.

Integridad de los datos: las reglas de negocio definidas se cumplen en todos los casos: el cliente solo ve y modifica sus propios datos, los precios históricos de las facturas no se alteran si un producto cambia, los estados de órdenes avanzan únicamente en la dirección establecida, y el stock nunca permite agregar al carrito más unidades de las disponibles.

Notificaciones en tiempo real: cuando el mecánico actualiza el estado de una orden de servicio, el cliente propietario recibe la notificación en su dispositivo sin necesidad de refrescar manualmente la aplicación.

Validación en dispositivo real: cada funcionalidad ha sido probada en hardware físico, no únicamente en emulador, garantizando que el comportamiento de la cámara, el GPS y las notificaciones responde a condiciones reales de uso.

Propuesta de valor y caso de negocio

Propuesta, funcionalidad y objetivo del prototipo

La propuesta de Taller Mecánico es reemplazar la coordinación manual (llamadas, WhatsApp, cuadernos) entre un taller mecánico y sus clientes por una aplicación móvil que cubre todo el ciclo de vida de una visita al taller desde que el cliente decide pedir un servicio o comprar un repuesto, hasta que el taller entrega el vehículo y factura y que además le da al dueño del negocio un panel de control en tiempo real sobre su operación. A continuación, se describe la funcionalidad de cada módulo tal como quedó implementada.

Autenticación y gestión de sesión

El cliente puede registrarse con sus datos personales, iniciar sesión de forma segura mediante un token JWT con tiempo de expiración, y cerrar sesión desde cualquier punto de la aplicación. El rol de "cliente" se asigna automáticamente al momento de crear la cuenta mediante un disparador en la base de datos.

El administrador y el mecánico comparten el mismo flujo de autenticación, con la diferencia de que al iniciar sesión el sistema detecta su rol y habilita automáticamente las funciones de gestión que el cliente no tiene visibles.

Inicio

El cliente puede ver la pantalla principal del taller con su nombre, imagen y mensaje de bienvenida, accesos directos para llamar, escribir por WhatsApp o abrir la ubicación en el mapa, una sección informativa del negocio, las promociones vigentes y una galería de fotos del taller.

El administrador cuenta con el control total de este módulo: puede editar la imagen principal del taller ya sea desde la galería del celular o mediante una URL, modificar el texto informativo, y gestionar las promociones y la galería de fotos de forma completa, pudiendo crear, editar, activar, desactivar o eliminar cada elemento, todo sin necesidad de acceder directamente a la base de datos.

Catálogo, carrito y compras

El cliente puede explorar el catálogo de productos, filtrarlo por categoría o por búsqueda de texto, y verificar la disponibilidad de stock antes de agregar artículos al carrito. Desde la misma tarjeta del producto puede ajustar la cantidad o eliminarlo sin necesidad de entrar al carrito. Al finalizar la compra, el sistema calcula automáticamente el ITBMS (7%) y genera una factura con numeración correlativa. El cliente puede consultar su historial de pedidos y facturas en cualquier momento desde su perfil.

El administrador tiene acceso a controles adicionales sobre el catálogo: puede crear nuevos productos, editarlos, eliminarlos, ajustar el stock manualmente, y gestionar las categorías disponibles, incluyendo la opción de activarlas o desactivarlas según la necesidad del taller.

Solicitud y seguimiento de servicio

El cliente puede solicitar un servicio completando un formulario donde registra los datos de su vehículo placa, marca, modelo, año, color y kilometraje, selecciona los servicios requeridos de un checklist de 12 opciones, adjunta una fotografía tomada desde la cámara del dispositivo y añade observaciones opcionales. Una vez enviada la solicitud, puede hacer seguimiento del estado de su orden en una línea de tiempo que avanza desde Pendiente hasta Entregado, con notificaciones en tiempo real cada vez que el mecánico realiza un cambio.

El administrador y el mecánico ven en este módulo la lista completa de órdenes de servicio activas con el estado de cada vehículo. Desde esta vista pueden avanzar el estado de cada orden y generar presupuestos asociados, sin necesidad de llamar al cliente ni actualizar nada de forma manual.

Perfil del cliente

Datos personales editables (nombre, teléfono, dirección; el correo queda de solo lectura por ser el mismo usado para iniciar sesión), con un modo de solo lectura por defecto y un botón "Editar" explícito para evitar cambios accidentales. Desde el perfil también se accede al historial de servicios, al carrito, a los pedidos, a las facturas, y solo si el usuario tiene rol de administrador o mecánico a un botón adicional que abre el panel de administración.

Panel de administración (mismos módulos, controles ampliados)

El administrador y el mecánico tienen acceso a todos los elementos anteriores y además visualizan un botón adicional que abre el panel de administración, centralizado en este mismo punto de la app para no duplicar pantallas.

Dashboard

Este módulo es exclusivo para administrador y mecánico. Presenta un panel con gráficos de barras desarrollados con Canvas de Compose, sin depender de librerías externas que muestran: pedidos por estado, órdenes de servicio por estado, ventas mensuales por número de facturas y monto total, productos más vendidos, y alertas automáticas de productos cuyo stock está por debajo del umbral configurado.

Notificaciones en tiempo real

Cada vez que el mecánico actualiza el estado de una orden de servicio, el cliente propietario del vehículo recibe una notificación local en su dispositivo de forma casi inmediata, gracias a un canal de Supabase Realtime que escucha los cambios en la tabla de solicitudes. Esto elimina la necesidad de que el cliente llame para preguntar por su carro.

Sector comercial al que va dirigido

El proyecto se dirige al sector de servicios automotrices específicamente talleres mecánicos independientes y de mediana escala en Panamá dedicados a mantenimiento preventivo, reparación general y venta de repuestos al detalle. Este segmento se caracteriza por una alta informalidad operativa, es decir, la mayoría de los talleres de este tamaño no cuentan con ningún sistema digital, gestionan citas y reparaciones por teléfono o WhatsApp, y llevan el inventario en cuadernos o en el mejor de los casos en hojas de cálculo sueltas. Las soluciones de software existentes para el sector automotriz (ERPs especializados, sistemas de concesionarias) están diseñadas y tarifadas para cadenas grandes, dejando un vacío para el taller pequeño e independiente que es exactamente el segmento donde compite Taller Mecánico, con foco en la simplicidad de uso y cero costos de entrada para el nivel de escala que resuelve el proyecto.

Público al que está orientado

Se identifican dos perfiles de usuario final, con necesidades distintas pero atendidas por la misma aplicación:

Perfil	Descripción	Necesidad principal
Administrador o mecánico del taller	Dueño o encargado operativo de un taller pequeño o mediano, con conocimiento técnico automotriz pero no necesariamente alta alfabetización digital.	Ver de un vistazo el estado de todos los vehículos y del inventario, sin tener que preguntar o buscar en papel.
Cliente propietario de vehículo	Persona adulta con vehículo propio que necesita mantenimiento periódico o reparaciones, acostumbrada a resolver trámites cotidianos desde su teléfono.	Saber en qué va su reparación y comprar repuestos sin tener que llamar o ir presencialmente a preguntar.

Beneficios para la sociedad, el público y el sector comercial

Beneficio social

Empuja a un sector tradicionalmente informal hacia una operación más trazable en donde cada servicio queda registrado con fecha, estado y responsable, lo que reduce el espacio para cobros no justificados o falta de seguimiento.

Facilita el acceso a talleres formales para personas con movilidad reducida o poco tiempo disponible, al eliminar la necesidad de visitas presenciales solo para consultar un estado.

Sienta un precedente replicable de digitalización de bajo costo para microempresas de servicios, un problema compartido por muchos sectores informales del país.

Beneficio para el público (cliente final)

Transparencia: el cliente ve el estado real de su vehículo y el detalle de lo que se le va a cobrar (presupuesto por la línea de repuesto y mano de obra) antes de que se ejecute el trabajo.

Ahorro de tiempo: no necesita llamar para preguntar "¿cómo va mi carro?" ni desplazarse para comprar un repuesto simple.

Historial permanente: facturas, pedidos y solicitudes de servicio quedan disponibles indefinidamente, útil para garantías o reclamos futuros.

Beneficio comercial (el negocio/taller)

Visibilidad gerencial inmediata: el dueño del taller ve en un dashboard qué se vendió, qué está pendiente y qué producto se está agotando, sin tener que pedirle el dato a nadie.

Reducción de carga operativa: menos llamadas repetidas de clientes preguntando por su vehículo, porque la app ya les responde eso.

Canal de venta adicional: el catálogo de repuestos funciona como una tienda dentro de la misma app, generando ventas que hoy dependen de que el cliente pase físicamente por el mostrador.

Costo del proyecto

El costo de infraestructura operativa durante esta fase es de \$0/mes, dado que la solución está construida íntegramente sobre herramientas en capa gratuita. Esta decisión responde a una estrategia deliberada: la fase actual representa un período de prueba en producción real, orientado a validar la adopción del sistema por parte del taller y sus clientes antes de comprometer inversión en infraestructura escalable.

El costo tangible del proyecto en esta etapa corresponde al tiempo invertido por el equipo de desarrollo, estimado en aproximadamente \$1,430 en horas de trabajo (ver detalle completo en la sección de gestión de costos). Una vez superada la fase de validación y confirmado el crecimiento en volumen de usuarios y operaciones, se contempla la migración hacia un plan de infraestructura de mayor capacidad, convirtiendo lo que hoy es un costo cero en una inversión justificada por datos reales de uso.

Declaración de alcance y especificación funcional

Incluido en el alcance

- **Autenticación:** registro de usuarios, inicio de sesión con token JWT y expiración automática de sesión, y cierre de sesión explícito desde cualquier punto de la app.
- **Módulo Cliente:** pantalla de inicio informativa del taller, catálogo de productos con carrito y checkout que incluye cálculo automático del ITBMS y generación de factura, solicitud de servicio con checklist de trabajos y evidencia fotográfica, seguimiento del estado de la orden en línea de tiempo, e historial de pedidos, facturas y perfil editable.
- **Módulo Administrador / Mecánico:** reutiliza las mismas pantallas del cliente y las extiende con controles de gestión: manejo de stock, alta, edición y baja de productos y categorías, gestión de órdenes de servicio activas, edición completa del contenido de la pantalla de inicio (imagen, textos, promociones y galería), y acceso al Dashboard con reportes gráficos de ventas, pedidos y alertas de inventario.

- **Notificaciones en tiempo real:** el cliente recibe una notificación local en su dispositivo cada vez que el mecánico actualiza el estado de su orden, mediante Supabase Realtime.
- **Uso de sensores del dispositivo:** cámara y galería para evidencia fotográfica e imágenes de contenido; GPS para registro de ubicación en el check-in del servicio.
- **Consumo de API REST:** integración explícita con los cuatro verbos HTTP —GET, POST, PUT y PATCH— a través de Retrofit, aplicada en el módulo de gestión de categorías.

Fuera del alcance

- **Pasarela de pago real:** la aplicación genera facturas con cálculo de ITBMS incluido, pero el procesamiento de cobros electrónicos mediante tarjetas o transferencias queda fuera de esta versión. El pago se gestiona por los medios que el taller ya utiliza actualmente.
- **Versión web e iOS:** el prototipo está desarrollado exclusivamente para Android. Una expansión multiplataforma se contempla como evolución natural del producto una vez superada la fase de validación.
- **Pruebas automatizadas exhaustivas:** dada la restricción de tiempo del proyecto, no se implementan suites de pruebas unitarias o instrumentadas. La validación funcional se realiza de forma manual sobre dispositivo real.

Especificación funcional detallada por módulo

Esta sección documenta las reglas de negocio concretas implementadas en el código, tal como serán evaluadas en la sustentación técnica.

Autenticación y sesión

Toda cuenta nueva se crea automáticamente con rol "cliente" mediante un disparador en base de datos (`handle_new_user`) al momento del registro. El token de sesión tiene una vigencia de 3,600 segundos según lo devuelve Supabase Auth; este valor se guarda localmente junto al token y, al abrir la aplicación, si el tiempo ya fue superado, la sesión se descarta y se exige un nuevo inicio de sesión.

Vehículos

Las placas se validan contra el patrón nacional: entre 1 y 3 letras seguidas de 3 a 6 dígitos, con guion opcional, cubriendo placas particulares, comerciales y oficiales. El año del vehículo debe estar entre 1980 y el año siguiente al actual. Marca, modelo y color se validan para evitar datos basura. Un vehículo solo puede eliminarse si no tiene solicitudes de servicio asociadas.

Carrito de compras

Cada producto ocupa una única fila por usuario en base de datos; agregar un artículo ya presente incrementa su cantidad en lugar de duplicar la línea. Reducir la cantidad a cero elimina la línea automáticamente. La cantidad máxima agregable está limitada al stock disponible en ese momento.

Pedidos y facturación

El ITBMS se calcula al 7% sobre el subtotal. Cada factura recibe un número correlativo autogenerado e irreplicable. Los pedidos siguen el flujo: Por atender → En preparación → Listo para retirar → Entregado, con posibilidad de pasar a Cancelado. El cliente únicamente puede cancelar sus propios pedidos; no puede marcarlo como entregado ni modificar al personal asignado. Las líneas de pedido guardan una copia del precio y descripción del producto al momento de la compra, garantizando que facturas históricas no se alteren si el producto cambia o se elimina posteriormente.

Servicios (solicitudes de reparación)

El checklist ofrece 12 opciones fijas configuradas en base de datos; una solicitud puede incluir una o varias. Toda solicitud nueva inicia en estado Pendiente sin excepción, y los estados avanzan en una única dirección: Pendiente → Cita programada → Vehículo recibido → En reparación → Listo para entregar → Entregado. El presupuesto asociado a una solicitud se compone de líneas independientes de tipo *Repuesto* o *Mano de obra*, gestionadas exclusivamente por el personal del taller.

Productos e inventario

Cada producto tiene un umbral de stock configurable; al alcanzarlo o caer por debajo, aparece automáticamente en la sección de Stock Bajo del dashboard. Desactivar un producto desde la lista lo deshabilita sin eliminarlo, preservando la integridad de pedidos y facturas anteriores. El borrado real de la fila requiere una acción explícita con confirmación en pantalla. Todo producto debe tener una categoría asignada; las categorías se administran de forma independiente sin afectar los productos ya asociados a ellas.

Contenido de Inicio

La imagen principal, las promociones y la galería de fotos pueden cargarse desde la galería nativa del dispositivo o mediante una URL directa. Tanto las promociones como las fotos de galería siguen el mismo esquema de gestión: desactivación sin borrado mediante un switch, y eliminación real con confirmación explícita cuando se requiere.

Registro de requisitos

Requisitos derivados del caso de negocio. Cada uno se referencia por su ID en la matriz de trazabilidad.

ID	Requisito	Tipo	Fuente	Prioridad
REQ-01	Pantalla de Login	Funcional	Técnico	Muy alta
REQ-02	Pantalla de registro de usuario nuevo	Funcional	Técnico	Muy alta
REQ-03	Seguridad JWT con expiración de sesión	No funcional	Técnico	Muy alta
REQ-04	Token como mecanismo de seguridad del API	No funcional	Técnico	Muy alta
REQ-05	Cierre de sesión (Logout)	Funcional	Técnico	Alta
REQ-06	Funcionalidad de negocio bajo el objetivo diseñado	Funcional	Técnico	Muy alta
REQ-07	Dashboard con informes y gráficos	Funcional	Técnico	Alta
REQ-08	API con GET/POST/PUT/PATCH vía Retrofit/OkHttp/Ktor	No funcional	Técnico	Alta
REQ-09	Uso de al menos un sensor del dispositivo	No funcional	Técnico	Alta
REQ-10	Catálogo de productos con carrito de compras	Funcional	Negocio	Alta
REQ-11	Checkout con factura e ITBMS	Funcional	Negocio	Alta
REQ-12	Solicitud de servicio con checklist y foto	Funcional	Negocio	Alta
REQ-13	Seguimiento de servicio en tiempo real	Funcional	Negocio	Alta
REQ-14	Perfil de cliente editable	Funcional	Negocio	Media
REQ-15	Panel de administración de productos/categorías/servicios	Funcional	Negocio	Alta

REQ-16	Edición de contenido de Inicio (imagen, texto, promociones, galería)	Funcional	Negocio	Media
---------------	--	-----------	---------	-------

Registro de interesados

Interesado	Interés	Influencia	Estrategia
Cliente final (dueño de vehículo/ comprador)	Servicio rápido y transparente	Alta — usuario final	UX simple, seguimiento en tiempo real
Dueño / administrador del taller	Visibilidad y control del negocio	Alta — decide adopción	Panel de administración y reportes
Mecánico / staff operativo	Registrar y actualizar servicios rápido	Media	Flujo de check-in y timeline simplificado
Profesor / evaluador (Jorge Cisneros)	Cumplimiento de la guía del curso	Alta — califica el proyecto	Trazabilidad de requisitos en este documento
Equipo de desarrollo	Aprendizaje y nota del curso	Alta — ejecuta el proyecto	Scrum con roles y ceremonias definidas

Estructura de desglose del trabajo (EDT)

#	Paquete de trabajo	Entregable	Responsable(s)
1	Base de datos y seguridad	Esquema PostgreSQL + políticas RLS en Supabase	Fernando Lezcano
2	Autenticación y sesión	Login, registro, JWT con expiración, logout	Fernando Lezcano
3	Módulo cliente: catálogo y compras	Catálogo, carrito, checkout, facturas, pedidos	Edward Camaño

4	Módulo cliente: servicios	Solicitud de servicio (foto + GPS), seguimiento	Emanuel González
5	Módulo administrador	Panel de administración integrado a Productos/Servicios, Dashboard con reportes	Emanuel González
6	API REST explícito (Retrofit)	Módulo de categorías con GET/POST/PUT/PATCH	Fernando Lezcano
7	Edición de contenido de Inicio	Imagen, texto, promociones y galería editables	Fernando Lezcano
8	QA y documentación	Pruebas manuales, plan del proyecto, material de sustentación	Edward Camaño, Emanuel González, María Madrid

Equipo del proyecto y matriz RACI

Roles

Integrante	Rol	Responsabilidad principal
Gonzalo Hooker	Product Owner	Prioriza el backlog, define criterios de aceptación y valida cada incremento
Edwin Hou	Scrum Master	Facilita ceremonias, da seguimiento al sprint y remueve impedimentos
Fernando Lezcano	Full Stack	Base de datos, seguridad, módulo administrador e integración Retrofit
Edward Camaño	Front End / QA	Pantallas de catálogo/compras y pruebas manuales de esos flujos

Emanuel González	Front End / QA	Pantallas de solicitud/módulo de administrador/seguimiento de servicios, modulo administrador y pruebas manuales
María Madrid	Documentación y soporte QA	Redacción de este plan, material de sustentación y apoyo en pruebas

Matriz RACI

R = Responsable (ejecuta) · A = Aprueba (rinde cuentas) · C = Consultado · I = Informado

Paquete de trabajo	E. Hoo	G. Hooke	F. Lezcano	E. Camacho	E. González	M. Madrid	Paquete de trabajo
Base de datos y seguridad	C	I	R/A	I	I	I	Base de datos y seguridad
Autenticación y sesión	C	I	R/A	C	C	I	Autenticación y sesión
Catálogo y compras	A	I	C	R	C	I	Catálogo y compras
Solicitud y seguimiento de servicios	A	I	C	C	R	I	Solicitud y seguimiento de servicios
Módulo administrador / Dashboard	A	I	R	C	C	I	Módulo administrador / Dashboard

Gestión del proyecto: Metodología Scrum

Esta sección describe cómo se orquestó el proyecto desde la perspectiva de gestión qué decisiones se tomaron, cómo se organizó el trabajo y cómo se dio seguimiento mientras avanzaba el proyecto.

Por qué Scrum

Se eligió Scrum por tres razones concretas del contexto del proyecto:

1. El plazo era corto y fijo, por lo que convenía entregar valor de forma incremental en vez de intentar un único gran lanzamiento al final.
2. El alcance tenía incertidumbre razonable al inicio, algo que Scrum maneja mejor que un enfoque en cascada.
3. El equipo, de 6 personas, es del tamaño ideal para un solo Scrum Team sin necesidad de escalar a múltiples equipos.

Roles Scrum

Product Owner: Gonzalo Hooker — priorizó el backlog y validó cada incremento contra las necesidades del negocio.

Scrum Master: Edwin Hou — facilitó las ceremonias y removió impedimentos del equipo.

Development Team: Fernando Lezcano (Full Stack), Edward Camaño (Front End/QA), Emanuel González (Front End/QA), María Madrid (Documentación y soporte QA).

Ceremonias aplicadas

Sprint Planning: al inicio de cada sprint, para seleccionar y estimar historias del backlog.

Daily Scrum: reunión corta diaria (vía WhatsApp/Discord) para reportar avance, plan del día e impedimentos.

Sprint Review: al cierre de cada sprint, demostración del incremento funcional al Product Owner.

Sprint Retrospective: al cierre de cada sprint, para ajustar el proceso del siguiente.

Definition of Done

Una historia de usuario se considera terminada solo si cumple todos estos criterios:

El código compila sin errores (`./gradlew assembleDebug` exitoso).

La funcionalidad se probó manualmente en un dispositivo o emulador real, no solo revisada en el editor.

No introduce una regresión visible en una funcionalidad ya aceptada en un sprint anterior.

El Product Owner la revisó y aceptó en la Sprint Review correspondiente.

Product Backlog priorizado (muestra representativa)

Prioridad	Historia de usuario	Estimación	Sprint
Muy alta	Como cliente, quiero registrarme e iniciar sesión de forma segura	5 pts	Planificación / Sprint 1
Muy alta	Como cliente, quiero ver el catálogo y agregar productos a mi carrito	8 pts	Sprint 1
Alta	Como cliente, quiero pagar mi carrito y recibir una factura	8 pts	Sprint 1
Alta	Como cliente, quiero solicitar un servicio con foto y checklist	8 pts	Sprint 1
Alta	Como cliente, quiero ver el estado de mi servicio en tiempo real	5 pts	Sprint 1
Media	Como cliente, quiero editar mis datos de perfil	3 pts	Sprint 2
Media	Como administrador, quiero gestionar productos, stock y categorías	8 pts	Sprint 2
Media	Como administrador, quiero ver reportes y gráficos de mi negocio	5 pts	Sprint 2
Media	Como administrador, quiero ver y actualizar el estado de los vehículos	5 pts	Sprint 2
Media	Como administrador, quiero editar el contenido de Inicio (fotos y texto)	5 pts	Sprint 2
Media	Como equipo, queremos demostrar un API REST con Retrofit explícito	5 pts	Sprint 2

Narrativa sprint a sprint: planeado vs. entregado

Sprint 1 (2 – 8 de julio)

Planeado: autenticación completa, catálogo con carrito, checkout con factura, solicitud de servicio con checklist y foto, y seguimiento del estado del servicio.
Entregado: el 100% de lo planeado, sin recorte de alcance.

Sprint 2 (9 – 12 de julio)

Planeado: perfil editable, panel de administración (productos, categorías, servicios), Dashboard con reportes, edición de contenido de Inicio, y el módulo Retrofit explícito. Entregado: el 100% de lo planeado, más ajustes surgidos en Sprint Review con el Product Owner (unificar Inventario dentro de Productos, extender Retrofit a los 4 verbos) ver detalle en el Registro de decisiones (sección 15).

Plan de ejecución del proyecto

Mientras la sección anterior describe el marco de trabajo (Scrum), esta describe la mecánica concreta con la que se ejecutó el trabajo día a día.

Entorno y herramientas

IDE	Android Studio / IntelliJ IDEA, sobre Kotlin + Jetpack Compose
Backend	Supabase (Postgres, Auth, Storage, Realtime), capa gratuita
Cliente HTTP adicional	Retrofit + json, para el módulo de categorías
Dispositivos de prueba	Emulador Android y un dispositivo físico (Honor) por USB/ADB
Build	Gradle (./gradlew assembleDebug), JDK 21 (JBR de IntelliJ)

Flujo de ejecución de cada historia

Cada historia de usuario sigue un flujo de trabajo definido que garantiza que ninguna funcionalidad se considere terminada hasta que haya sido probada en condiciones reales.

1. Selección: el Development Team toma la siguiente historia disponible del Sprint Backlog, respetando el orden de prioridad establecido por el Product Owner. Nadie elige libremente qué trabajar; la prioridad ya está definida.

2. Implementación: el desarrollo sigue el patrón arquitectónico establecido para el proyecto: Repository → ViewModel → Screen. Esto garantiza que todas las historias se construyen de forma consistente y que la lógica de negocio nunca queda mezclada con la interfaz visual.

3. Compilación e instalación: una vez implementada, la historia se compila ejecutando `./gradlew assembleDebug` y el APK resultante se instala en un dispositivo físico real mediante ADB. Que el código compile sin errores no es suficiente para darla por terminada; debe funcionar en hardware real.

4. Validación: el desarrollador responsable prueba manualmente la historia contra su criterio de aceptación definido. Si no cumple exactamente lo que la historia especifica, regresa a implementación.

5. Aceptación formal: la historia validada se presenta en la Sprint Review. Solo tras la aceptación explícita del Product Owner se considera oficialmente completada y se cierra.

Gestión de cambios durante la ejecución

Cualquier ajuste de alcance identificado durante el transcurso de un sprint como la decisión de fusionar la pantalla de Inventario dentro del módulo de Productos no se implementaba directamente. Primero se discutía con el Product Owner, se evaluaba su impacto sobre las historias en curso y, una vez aprobado, quedaba registrado en la bitácora de decisiones antes de ejecutarse. Este proceso garantiza que ningún cambio de alcance, por pequeño que parezca, quede sin trazabilidad ni aprobación formal, manteniendo la coherencia entre lo planificado y lo entregado.

Cronograma del proyecto

Fase	Fechas	Objetivo	Entregable
Planificación	29 jun – 1 jul	Definir backlog, diseñar base de datos y arquitectura	Backlog priorizado, esquema de base de datos
Sprint 1	2 – 8 jul	Autenticación, catálogo/carrito/checkout, solicitud y seguimiento de servicios	Incremento funcional: flujo completo del cliente
Sprint 2	9 – 12 jul	Perfil editable, panel de administración integrado, Dashboard con reportes, edición de Inicio, módulo Retrofit	Incremento funcional: flujo completo administrador + pulido general

Cierre y sustentación	13 – 15 jul	Auditoría de cumplimiento, redacción de este documento, preparación de la presentación	Plan del proyecto, presentación, demo en vivo
-----------------------	-------------	--	---

Presupuesto y plan de gestión de costos

Presupuesto detallado

Costo de oportunidad del equipo, con tarifas de referencia del mercado panameño para desarrollo junior:

Rol	Horas estimadas	Tarifa de referencia (USD/h)	Subtotal (USD)
Full Stack (Fernando Lezcano)	45	\$10	\$450
Front End / QA (Edward Camaño)	30	\$8	\$240
Front End / QA (Emanuel González)	30	\$8	\$240
Product Owner (Edwin Hou)	20	\$9	\$180
Scrum Master (Gonzalo Hooker)	20	\$9	\$180
Documentación / QA (María Madrid)	20	\$7	\$140
Total estimado	165	—	\$1,430

Costo de infraestructura, con tarifas de referencia del mercado de las herramientas implementadas:

Costo de infraestructura (prototipo / demo)	\$0/mes (capa gratuita de Supabase)
--	-------------------------------------

Costo de infraestructura (producción, referencia)	Supabase Pro desde \$25/mes si el volumen excede la capa gratuita
Costo de publicación en tienda	Cuenta de desarrollador Google Play: pago único de \$25
Costo total estimado (desarrollo + primer año)	≈ \$1,430 + \$300 (Supabase Pro anual) + \$25 (Play Store) ≈ \$1,755

Plan de gestión de costos

Método de estimación: las horas por historia de usuario se estimaron mediante analogía y juicio experto del equipo, asignando esfuerzo según la complejidad relativa de cada tarea. Dado el tamaño reducido del equipo y el tiempo disponible, no se aplicó planning poker formal, pero la estimación fue revisada y validada colectivamente antes de comprometerse con ella.

Línea base de costo: la referencia de control es la tabla de horas-persona por rol definida en la sección 12.1, aprobada por el Product Owner al inicio de la planificación. Cualquier desviación se mide contra esta línea base, no contra estimaciones informales posteriores.

Control de costos: en cada retrospectiva de sprint, el Scrum Master compara el avance real contra lo estimado. Si la variación en horas de algún paquete de trabajo supera el 20%, se abre una conversación formal con el Product Owner para decidir entre dos opciones: recortar funcionalidades de baja prioridad del backlog o redistribuir horas desde otro paquete con menor presión.

Aprobación de cambios de costo: cualquier decisión que implique un gasto real — como migrar del plan gratuito de Supabase a un plan de pago— requiere aprobación explícita del Product Owner antes de ejecutarse. Ningún integrante del equipo puede tomar esa decisión de forma unilateral.

Reserva de contingencia: dado que la infraestructura opera en capa gratuita, no existe exposición a sobrecostos de operación durante esta fase. La única contingencia relevante es de tiempo, y se gestiona reduciendo el alcance de las historias de menor prioridad en el backlog, conforme a lo establecido en el registro de riesgos.

Registro de riesgos

Riesgo	Prob.	Impacto	Mitigación
--------	-------	---------	------------

Tiempo insuficiente para cubrir todo el alcance	Media	Alto	Scrum con sprints cortos y backlog priorizado por valor de negocio
Dependencia de un proveedor externo (Supabase) fuera de servicio	Baja	Alto	Uso de capa gratuita documentada; arquitectura por repositorios que aislaría un futuro cambio de backend
Interpretación ambigua del requisito de "API propio"	Media	Medio	Módulo de categorías con Retrofit explícito como evidencia adicional; se documenta el razonamiento en este plan
Falta de pruebas automatizadas por restricción de tiempo	Alta	Medio	Pruebas manuales dirigidas por rol de QA en cada sprint review

Registro de problemas

Problemas técnicos reales detectados durante el desarrollo y su resolución (issue log), distinto del registro de riesgos porque aquí se documentan hechos que ya ocurrieron, no posibilidades futuras.

ID	Descripción	Severidad	Estado	Resolución
ISS-01	SDK path de Android apuntaba a la máquina de otro integrante (local.properties)	Baja	Resuelto	Corregido apuntando al SDK local propio; se documentó no compartir ese archivo
ISS-02	Gradle usaba Java 8 del sistema en vez de un JDK 17+	Media	Resuelto	Se fijó org.gradle.java.home a un JBR 21 estable en gradle.properties
ISS-03	Advertencia falsa de "opt-in InternalSerializationApi" en clases @Serializable	Baja	No aplica	Confirmado como falso positivo del analizador del IDE; no afecta la compilación real
ISS-04	El rol de usuario siempre se leía como "cliente" al iniciar sesión, sin importar la base de datos	Alta	Resuelto	Un campo no-nulable en el modelo Kotlin (teléfono) rompía la decodificación del

				perfil; se corrigió a nullable
ISS-05	No existía forma de crear una categoría nueva desde la app	Media	Resuelto	Se agregaron endpoints POST y PATCH explícitos vía Retrofit
ISS-06	El botón "Avanzar a: Listo para retirar" no cambiaba el estado del pedido	Alta	Resuelto	La restricción CHECK de la columna estado en Supabase estaba corrupta; se recreó limpia por SQL
ISS-07	La pantalla de Inventario duplicaba funcionalidad ya cubierta en Productos	Baja	Resuelto	Se fusionó Inventario dentro de la pestaña Productos del panel de administración

Registro de decisiones

Decisiones de diseño y alcance tomadas durante el proyecto (distinto del registro de problemas: aquí no hubo un error que corregir, sino una elección entre alternativas válidas).

Fecha	Decisión	Justificación
1 jul	Usar Supabase como backend en vez de construir un servidor propio desde cero	Restricción de tiempo del proyecto; Supabase ya provee Auth, Postgrest y Storage listos para usar
9 jul	Fusionar la pantalla de Inventario dentro de la pestaña Productos del administrador	Evitar pantallas duplicadas con la misma información, mismo patrón admin-sobre-la-misma-pantalla usado en el resto de la app
11 jul	Extender el módulo de Retrofit (Categorías) para cubrir los 4 verbos HTTP explícitamente	Reforzar el punto de "API propio" de la guía del curso con evidencia de código verificable
13 jul	Reutilizar el bucket de Storage "productos" para las imágenes de Inicio en vez de crear uno nuevo	Ese bucket ya tenía las políticas correctas (solo staff escribe, lectura pública); evita una migración SQL adicional

14 jul	Quitar el botón "Nuevo check-in" del Dashboard sin borrar la pantalla ni su código	El flujo de check-in manual ya no es la única puerta de entrada; se conserva el código por si se necesita un acceso distinto más adelante
14 jul	Postergar la creación del repositorio Git hasta terminar todos los cambios funcionales	Evitar commits intermedios desordenados; se documenta como pendiente explícito en el checklist de cierre

Plan de comunicación

Canal	Propósito	Frecuencia	Responsable
WhatsApp / Discord	Daily Scrum y coordinación rápida	Diaria	Gonzalo Hooker (Scrum Master)
Reunión de Sprint Review	Demostrar el incremento al Product Owner	Al cierre de cada sprint	Edwin Hou (Product Owner)
Reunión de Retrospectiva	Ajustar el proceso del equipo	Al cierre de cada sprint	Gonzalo Hooker (Scrum Master)
Repositorio de código (GitHub)	Final del desarrollo	Al cierre del sprint	Fernando Lezcano/Emanuel Gonzalez/Edward Camaño

Arquitectura de software y patrones de diseño

Esta sección detalla las decisiones de arquitectura y los patrones de diseño realmente implementados en el código, citando los archivos donde se pueden verificar durante la evaluación técnica.

Vista general de la arquitectura

La aplicación está construida sobre una arquitectura en capas, patrón estándar del desarrollo Android moderno, cuyo principio central es que cada capa tiene una responsabilidad única y no conoce los detalles internos de las demás. Esto hace que el código sea más fácil de mantener, depurar y escalar.

Capa de presentación (UI): contiene todas las pantallas construidas con Jetpack Compose, organizadas bajo el paquete `ui/`. Su única responsabilidad es mostrar información y capturar las acciones del usuario. No contiene lógica de negocio ni accede directamente a datos; todo lo que necesita lo recibe del `ViewModel`.

Capa de estado y lógica (ViewModel): existe un `ViewModel` por pantalla, que actúa como intermediario entre la interfaz y los datos. Expone el estado de la pantalla mediante `StateFlow`, de forma que cualquier cambio se refleja automáticamente en la UI sin que esta tenga que pedirlo. También es el punto donde se procesan los eventos del usuario —un botón presionado, un formulario enviado— y se decide qué acción tomar.

Capa de acceso a datos (Repository): hay una clase `Repository` por entidad de negocio (productos, pedidos, vehículos, servicios, etc.). Su función es aislar al resto de la aplicación de los detalles de cómo se obtienen o persisten los datos. El `ViewModel` no sabe ni le importa si el dato viene de Supabase, de una caché local o de una API externa; solo le pide al `Repository` y este resuelve.

Capa de fuente de datos: es el nivel más bajo y el único que interactúa directamente con los servicios externos. La mayor parte del tráfico de datos pasa por el SDK de Supabase, que cubre autenticación (`Auth`), base de datos (`PostgREST`), almacenamiento de archivos (`Storage`) y escucha de cambios en tiempo real (`Realtime`). En el módulo de categorías se usa adicionalmente `Retrofit`, que consume el mismo API REST de Supabase pero de forma explícita con los cuatro verbos HTTP, como requerimiento técnico del proyecto.

Patrón MVVM (Model-View-ViewModel)

Cada pantalla de la aplicación sigue el mismo patrón de forma consistente, lo que garantiza que cualquier integrante del equipo pueda entender y modificar cualquier pantalla sin necesidad de aprender una estructura diferente en cada una.

El `ViewModel` es el núcleo de cada pantalla. Mantiene un `StateFlow` inmutable que representa el estado completo de lo que la pantalla debe mostrar en un momento dado. Por ejemplo, `CatalogoViewModel.kt` gestiona un objeto `CatalogoEstado` que contiene la lista de productos, el estado de carga, los filtros activos y cualquier error ocurrido. Este estado es la única fuente de verdad de la pantalla: si algo no está en el estado, la UI no lo muestra.

La pantalla Composable por ejemplo, `CatalogoScreen.kt` se suscribe a ese estado mediante `collectAsState()` y se redibuja automáticamente cada vez que el estado cambia, sin que el desarrollador tenga que indicarle explícitamente cuándo actualizarse. Jetpack Compose se encarga de eso por diseño.

El flujo de datos es estrictamente unidireccional: la UI nunca modifica el estado directamente. Cuando el usuario realiza una acción aplicar un filtro, agregar un producto al carrito, enviar un formulario la pantalla simplemente invoca una función del ViewModel. Es el ViewModel quien decide cómo procesar esa acción, actualiza el estado, y la UI reacciona al cambio. Esto elimina una clase entera de errores comunes donde la interfaz y los datos quedan fuera de sincronía, y hace que el comportamiento de cada pantalla sea predecible y trazable.

Patrón Repository

Cada entidad de negocio de la aplicación Producto, Pedido, Orden, Vehículo, Categoría, Promoción, ItemGalería, entre otras tiene su propia clase Repository ubicada bajo `data/repository/`. Su propósito es actuar como una capa de abstracción entre el ViewModel y los servicios externos, de forma que el resto de la aplicación nunca necesita saber cómo se obtienen los datos, solo que los puede pedir.

Cada Repository expone métodos con nombres propios del negocio: `listarActivos()`, `crear()`, `actualizar()`, `eliminar()`. Cuando el ViewModel llama a `listarActivos()` en el `ProductoRepository`, no sabe ni le importa si ese dato viene de una consulta PostgREST a Supabase, de una llamada Retrofit, o eventualmente de una caché local. Esa decisión queda encapsulada dentro del Repository y no se filtra hacia arriba.

Esto tiene una ventaja práctica concreta: si en el futuro se decide cambiar Supabase por otro proveedor, o agregar una capa de caché local, ese cambio se hace únicamente dentro del Repository correspondiente. El ViewModel y la pantalla no se tocan. El impacto de un cambio de infraestructura queda contenido en un solo lugar.

Singleton

`SupabaseClientProvider.kt` expone una única instancia del cliente de Supabase (lateinit var client, inicializada una sola vez) que reutilizan todos los repositorios de la aplicación.

Tipado de rutas con clases selladas (type-safe navigation)

La navegación entre pantallas (`Screen.kt`) se modela con una sealed class donde cada pantalla es un objeto con su propia ruta de texto, evitando errores de tipeo y centralizando en un solo archivo todas las pantallas posibles de la aplicación.

Inyección de dependencias manual (constructor injection)

En vez de introducir un framework de inyección de dependencias (Hilt, Koin), cada ViewModel recibe sus dependencias (Repositories) como parámetros de constructor con valor por defecto (@JvmOverloads), permitiendo sustituirlas en pruebas sin cambiar la clase.

Patrón Adaptador: doble camino de consumo del mismo API

CategoriaRepository.kt es el caso más claro de un patrón de diseño aplicado de forma deliberada en el proyecto. Desde afuera se comporta exactamente igual que cualquier otro Repository: expone los mismos métodos de negocio y el ViewModel lo consume sin ninguna diferencia aparente. Sin embargo, por dentro utiliza Retrofit puro como cliente HTTP en lugar del SDK de Supabase, apuntando al mismo API REST de PostgREST.

El patrón Adaptador es precisamente eso: una capa que traduce una implementación técnica concreta en este caso una integración HTTP diferente hacia una interfaz uniforme que el resto de la aplicación entiende. El ViewModel no sabe, ni necesita saber, que por debajo hay un cliente distinto. Solo llama al método y recibe el resultado. El cambio de implementación queda completamente invisible hacia arriba.

Seguridad: JWT + Row Level Security

La seguridad de la aplicación opera en dos niveles que trabajan juntos: uno en el cliente y otro directamente en la base de datos.

En el cliente, cuando el usuario inicia sesión, Supabase emite un JSON Web Token (JWT) que se guarda localmente en el dispositivo junto a su marca de tiempo de expiración. Cada vez que la aplicación arranca, verifica si ese token sigue vigente. Si el tiempo ya venció, la sesión se descarta y se exige un nuevo inicio de sesión. Esto impide que una sesión quede abierta indefinidamente en un dispositivo.

En la base de datos, ese mismo token viaja adjunto en cada solicitud que la aplicación hace a Supabase. Cuando la solicitud llega a Postgres, entra en juego el segundo nivel de seguridad: las políticas de Row Level Security (RLS). Estas políticas son reglas definidas directamente en la base de datos que determinan qué filas puede leer o modificar cada usuario según su identidad, extraída del token. Por ejemplo, un cliente solo puede ver sus propios pedidos y facturas, aunque técnicamente todos estén en la misma tabla.

La ventaja clave de este enfoque es que la seguridad no depende de que la aplicación filtre correctamente los datos antes de mostrarlos. Es Postgres quien toma esa decisión, en el nivel más bajo posible, lo que significa que incluso si hubiera un error

en la lógica de la app, la base de datos no entregaría datos que el usuario no tiene permitido ver.

Flujo de datos unidireccional (StateFlow → Compose)

Todas las pantallas de la aplicación siguen el mismo ciclo de vida de datos, sin excepción.

El ViewModel expone el estado de la pantalla como un StateFlow<Estado> inmutable, lo que significa que nadie fuera del ViewModel puede modificarlo directamente. La pantalla Composable se suscribe a ese flujo mediante collectAsState() y, cada vez que el estado cambia, Compose redibuja automáticamente solo los elementos afectados, sin que el desarrollador tenga que indicarle qué actualizar ni cuándo.

Cuando el usuario interactúa con la pantalla presiona un botón, escribe en un campo, selecciona un filtro la UI no toca el estado. En cambio, invoca una función del ViewModel, que procesa la acción, actualiza el estado internamente y el ciclo vuelve a comenzar. La dirección siempre es la misma: **estado baja hacia la UI, eventos suben hacia el ViewModel**.

Este enfoque elimina una fuente muy común de errores en interfaces complejas: el estado inconsistente, donde la pantalla muestra algo distinto a lo que realmente hay en memoria. Al existir una única fuente de verdad controlada por el ViewModel, lo que el usuario ve siempre refleja exactamente el estado real de la aplicación.

Modelo de datos

La base de datos (PostgreSQL, alojada en Supabase) organiza el dominio del negocio en 8 grupos de tablas:

Grupo	Tablas	Propósito
Usuarios y roles	perfiles	Extiende auth.users con nombre, teléfono, dirección y rol (cliente/mecánico/admin)
Catálogo e inventario	categorias, productos	Catálogo de repuestos, también funciona como inventario del mecánico
Carrito	carrito_items	Una fila por producto distinto en el carrito de cada usuario

Pedidos y facturas	pedidos, pedido_items, facturas	Compra de productos, sus líneas y la factura con numeración correlativa
Vehículos y servicios	vehiculos, tipos_servicio, solicitudes_servicio, solicitud_tipos, fotos_servicio, presupuesto_items	El expediente completo de una reparación: vehículo, checklist, fotos y presupuesto
Contenido de Inicio	config_taller, promociones, galeria	Información pública del taller, editable por el administrador desde la app
Notificaciones	notificaciones	Avisos al cliente cuando cambia el estado de su pedido o servicio
Vistas para reportes	vista_pedidos_por_estado, vista_servicios_por_estado, vista_ventas_por_mes, vista_stock_bajo, vista_productos_mas_vendidos	Datos agregados que alimentan el Dashboard, con seguridad heredada del usuario que consulta

Cada tabla del negocio tiene Row Level Security activado, con la regla general de que un cliente ve y modifica solo lo suyo, y el personal del taller ve y gestiona todo.

Sensores y hardware del dispositivo

GPS / ubicación

Para el registro de ubicación en el check-in de un vehículo, la aplicación utiliza FusedLocationProviderClient, la API de localización de Google Play Services.

A diferencia de acceder directamente al GPS del dispositivo, FusedLocationProviderClient combina inteligentemente múltiples fuentes de señalGPS, redes Wi-Fi y torres celulares y devuelve la ubicación más precisa disponible en ese momento, consumiendo la menor cantidad de batería posible. Esto lo hace más confiable que depender exclusivamente del GPS, que en espacios cerrados o con poca señal puede tardar o fallar.

En el contexto de la aplicación, cuando el cliente registra la llegada de su vehículo al taller, se captura automáticamente la coordenada del dispositivo en ese instante y se asocia a la solicitud de servicio, sin que el usuario tenga que ingresar nada manualmente.

Cámara y Galería

La aplicación utiliza dos mecanismos distintos para la captura y selección de imágenes, cada uno adecuado al contexto en que se usa.

Para capturar fotos en tiempo real como la evidencia fotográfica de un vehículo en la solicitud de servicio, el detalle de una orden, o la imagen de un producto se usa el contrato `TakePicturePreview()`. Este contrato abre directamente la cámara del dispositivo y devuelve la foto tomada a la aplicación de forma inmediata, sin necesidad de que el usuario gestione el archivo manualmente.

Para seleccionar imágenes existentes como la imagen principal del taller, las promociones o las fotos de la galería que el administrador gestiona desde la pantalla de Inicio se usa `PickVisualMedia()`, el selector nativo de fotos de Android. Esto permite al administrador elegir cualquier imagen ya almacenada en su dispositivo sin salir de la aplicación, con la misma experiencia visual que ofrece el sistema operativo.

En ambos casos, una vez obtenida la imagen, se sube a Supabase Storage, que actúa como servidor de archivos en la nube. La URL pública generada por Storage queda asociada a la fila correspondiente en la base de datos, de forma que la aplicación siempre referencia la imagen por su URL y no almacena archivos localmente.

Matriz de trazabilidad de requisitos

Vincula cada requisito con el módulo donde se implementó y el caso de prueba que lo valida.

#	Caso de prueba	Resultado esperado	Estado
1	Registro y login con credenciales válidas	Se crea la cuenta con rol cliente y se inicia sesión	Aprobado
2	Login con sesión expirada	La app pide iniciar sesión de nuevo en vez de dejar entrar	Aprobado
3	Agregar producto al carrito y ajustar cantidad	El panel +/- aparece y respeta el límite de stock	Aprobado
4	Checkout de un carrito con productos	Se genera un pedido y una factura con ITBMS correcto	Aprobado
5	Solicitar servicio con placa inválida	El formulario rechaza el envío y explica el formato esperado	Aprobado

6	Cambiar el estado de una orden como mecánico	El cliente dueño de la orden recibe una notificación	Aprobado
7	Ajustar stock manualmente desde Productos (admin)	El stock se actualiza de inmediato en la lista	Aprobado
8	Eliminar un producto con confirmación	Pide confirmar antes de borrar; cancelar no hace nada	Aprobado
9	Crear y desactivar una categoría (Retrofit)	La categoría nueva aparece; el switch la desactiva	Aprobado
10	Acceso al panel de administración con cuenta cliente	El botón "Panel de administración" no aparece	Aprobado
11	Editar perfil y cancelar sin guardar	Los datos vuelven a su valor original al cancelar	Aprobado
12	Compilación completa del proyecto	./gradlew assembleDebug finaliza en BUILD SUCCESSFUL	Aprobado
13	Editar Inicio: subir foto de galería y guardar texto	La imagen y el texto se actualizan y persisten al recargar	Aprobado

Checklist de calidad

Criterio de calidad	Cumple
El proyecto compila sin errores (BUILD SUCCESSFUL)	Sí
Cada pantalla sigue el patrón MVVM (Screen + ViewModel + StateFlow)	Sí
Toda tabla sensible del negocio tiene Row Level Security activado	Sí
No hay credenciales ni tokens escritos directamente en el código fuente	Sí
Los errores de red/base de datos se manejan con Result<T>, no se ignoran en silencio	Sí
Las validaciones de formularios cubren los campos obligatorios del negocio (placa, año, etc.)	Sí

Las imágenes subidas van a buckets de Storage con políticas de acceso restringidas por rol	Sí
Se probó cada funcionalidad en un dispositivo físico real, no solo en el emulador	Sí
Existen pruebas automatizadas (unitarias/instrumentadas)	No (ver Registro de riesgos)

Confirmación de cumplimiento del alcance

Verificación punto por punto de lo declarado como incluido en el alcance:

Ítem del alcance	Cumplido
Autenticación: registro, login con JWT y expiración, logout	Sí
Módulo Cliente completo (catálogo, carrito, checkout, servicios, timeline, perfil)	Sí
Módulo Administrador/Mecánico (productos, categorías, servicios, Dashboard)	Sí
Edición de contenido de Inicio (imagen, texto, promociones, galería)	Sí
Notificaciones en tiempo real de cambio de estado	Sí
Uso de sensores del dispositivo (GPS y cámara/galería)	Sí
API REST con los 4 métodos HTTP vía Retrofit explícito	Sí (módulo Categorías)

Todos los ítems declarados dentro del alcance fueron entregados y verificados mediante los casos de prueba. Los ítems explícitamente fuera de alcance no se implementaron por decisión consciente del equipo, no por omisión.

Informe final del proyecto

Mecanic Hou se planificó y ejecutó en 2 sprints de una semana cada uno (11 días efectivos), precedidos por 3 días de planificación. El equipo de 6 integrantes entregó el 100% de las historias comprometidas en ambos sprints, sin recorte de alcance, incorporando además 3 mejoras no planificadas originalmente que surgieron durante

las Sprint Reviews (fusión de Inventario en Productos, extensión de Retrofit a los 4 métodos, y edición completa del contenido de Inicio).

Durante el desarrollo se identificaron y resolvieron 7 problemas técnicos, ninguno de los cuales quedó pendiente al cierre. La aplicación final consta de más de 30 pantallas, 17 tablas de base de datos con seguridad a nivel de fila, y cubre los requisitos técnicos, además de las funcionalidades de negocio propias del prototipo.

Checklist de cierre

Ítem de cierre	Estado
Código fuente completo y compilando	Completo
Todos los requisitos funcionales del alcance entregados	Completo
Plan del proyecto	Completo
Aprobación del Product Owner sobre el incremento final	Completo
Lecciones aprendidas documentadas	Completo
Repositorio Git inicializado y subido a GitHub	Completo
App preparada	Completo

Conclusión

Mecanic Hou nace como respuesta a un problema concreto y vigente en el sector automotriz independiente de Panamá, es decir, la ausencia de herramientas digitales diseñadas específicamente para el flujo de trabajo de un taller mecánico de mediana escala. La solución no adapta un CRM genérico ni simplifica un ERP empresarial; construye desde cero una aplicación móvil cuya lógica, terminología y flujos responden exactamente a cómo opera este tipo de negocio.

El resultado es un prototipo funcional que cubre todo el ciclo de vida de una visita al taller: desde la solicitud de servicio y el seguimiento en tiempo real, hasta la venta de repuestos con facturación automática y un panel de control para el administrador. Todo esto operando sobre infraestructura de costo cero durante la fase de validación, lo que elimina la barrera de entrada para un negocio que históricamente no ha tenido acceso a tecnología a su medida.

Desde el punto de vista técnico, el proyecto valida una arquitectura sólida y mantenible: MVVM con flujo de datos unidireccional, patrón Repository que aísla las fuentes de datos, patrón Adaptador para el consumo explícito de API REST, y

seguridad gestionada en dos niveles mediante JWT y Row Level Security directamente en Postgres. Estas decisiones de diseño no son accesorias; son las que permitirán que el producto escale sin necesidad de reescribirse cuando el volumen de usuarios y operaciones crezca.

Las oportunidades de mejora identificadas durante el desarrollo integración de una pasarela de pago real, cobertura de pruebas automatizadas y expansión a plataformas adicionales no representan deudas técnicas no resueltas, sino un backlog priorizado y documentado para la siguiente iteración del producto. Mecanic Hou cierra esta fase como un sistema probado en dispositivo real, con roles diferenciados funcionando de principio a fin, listo para ingresar al período de validación con el cliente y sentar las bases de lo que puede convertirse en una solución replicable para el sector.

Lecciones Aprendidas

La nulabilidad de los datos importa más de lo que parece. Un tipo de dato declarado como no-nulable en el cliente que no coincide exactamente con la nulabilidad real de la columna en la base de datos puede romper silenciosamente una funcionalidad crítica sin lanzar ningún error visible. En este proyecto ocurrió precisamente con la lectura del rol de usuario: la aplicación no mostraba ningún fallo explícito, pero el comportamiento era incorrecto. La lección es clara: antes de modelar cualquier entidad en el cliente, vale la pena revisar explícitamente la definición de cada columna en la base de datos y asegurarse de que la nulabilidad coincide en ambos extremos.

Reutilizar pantallas entre roles es mejor que duplicarlas. La decisión de usar las mismas pantallas para el cliente y el administrador, agregando controles adicionales según el rol detectado, redujo significativamente el trabajo de mantenimiento y evitó que ambas experiencias divergieran con el tiempo. Cada vez que se corregía un error o se mejoraba algo en una pantalla, el beneficio aplicaba automáticamente a los dos roles. Duplicar pantallas habría duplicado también los problemas.

Definir el patrón arquitectónico desde el primer módulo acelera todo lo que viene después. Establecer desde el Sprint 1 la estructura Repository → ViewModel → Screen como esqueleto estándar de cada módulo permitió que los módulos siguientes se construyeran considerablemente más rápido. El equipo no discutía cómo organizar el código en cada historia; simplemente repetía el mismo patrón ya validado y se concentraba en la lógica específica de cada funcionalidad. La consistencia arquitectónica temprana es una inversión que se recupera rápidamente.

Cuando un dato parece correcto pero la base de datos lo rechaza, el problema suele estar en la restricción, no en el dato. Uno de los casos más confusos del proyecto fue un valor que visualmente parecía exactamente lo que la base de datos debía aceptar, pero era rechazado sistemáticamente. La causa resultó estar en la definición misma

del constraint, no en el dato enviado. Antes de asumir que el valor es incorrecto, vale la pena revisar cómo está definida la restricción, qué acepta exactamente y si hay diferencias de tipo, formato o codificación que no son evidentes a simple vista.